

Software Configuration Management(SCM)

Chapter 7

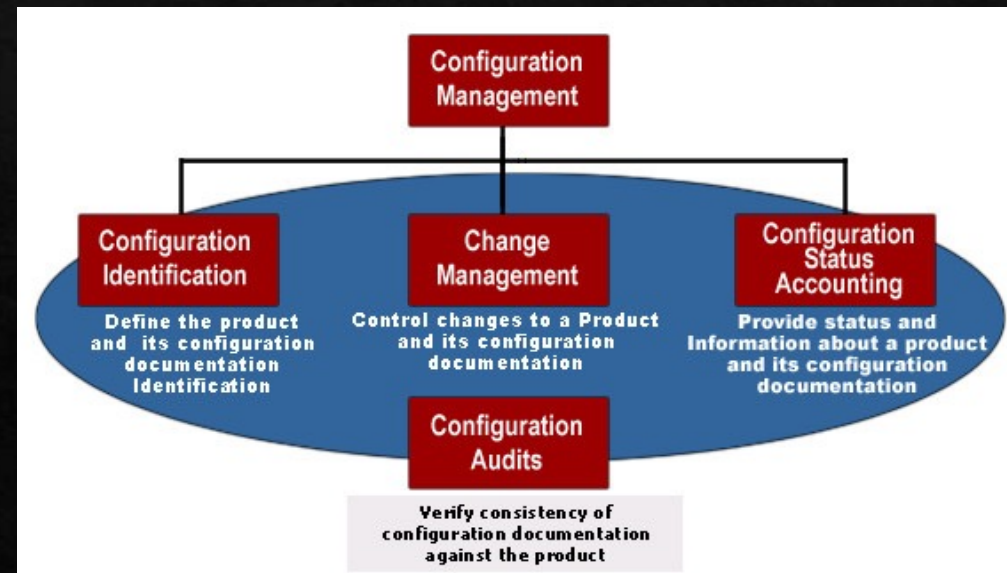
- ◆ **Concept**
- ◆ **Requirement and element of SCM**
- ◆ **Baseline**
- ◆ **SCM Repository**
- ◆ **Versioning and version control**
- ◆ **SCM Process**
- ◆ **Change control process**

Software Configuration Management(SCM)

- ◆ The art of identifying, organizing, and controlling modifications to the software being built.
- ◆ The main goal is to minimize mistakes.
- ◆ It is a umbrella activity, applied throughout the software engineering process.
- ◆ Auditing of the software configuration
- ◆ Reporting of all the changes applied to configuration
- ◆ Control of change

Requirement and element of SCM

- ◆ Identification, control, audit, and status accounting are the four basic requirements for a software configuration management system.
- ◆ **Identification:**
 - ◆ Each software part is labeled so that it can be identified.
 - ◆ Furthermore, there will be different versions of the software parts as they evolve over time, so a version or revision number will be associated with the part.
 - ◆ The key is to be able to identify any and all artifacts that compose a released configuration item.
 - ◆ It helps to answer the following question.
 - ◆ What are the component of the product?
 - ◆ What are the version of configured item?



Control

- ◆ In the context of configuration management, “control” means that proposed changes to a CI (Configuration Item) are reviewed and, if approved, incorporated into the software configuration.
- ◆ The goal is to make informed decisions and to acknowledge the repercussions associated with a change to the system.
- ◆ These changes may impact budgets, schedules, and associated changes to other components
- ◆ It answer the question:
 - ◆ Which item are controlled?
 - ◆ How are changes controlled?
 - ◆ Who controls the change?

Audit

- ◊ Auditing an SCM system means that approved requested changes have indeed been implemented.
- ◊ The audits allow managers to determine whether software evolution is proceeding both logically and in conformance with requirements for the software.
- ◊ The SCM system should document changes, versions, and release information for all components of each configuration item.
- ◊ When such documentation is in place, auditing becomes a straightforward analysis task.
- ◊ It verify the correctness of the product.

Status Accounting

- ◆ This accounting provides the historic information to determine both what happened and when on the software project.
- ◆ Status accounting enables the auditing requirement of the SCM.
- ◆ As a project manager, the status accounting holds a wealth of information on the amount of effort required throughout the life cycle of the product in its development and maintenance.
- ◆ This is critical to the software project manager in making estimates for new systems based on historic information.
- ◆ The SCM can be used as one of the key components of the project managers' metrics system.

Baseline

- ◇ A baseline is a formally accepted version of a software configuration item. It is designated and fixed at a specific time while conducting the SCM process. It can only be changed through formal change control procedures.
- ◇ **Activities during this process:**
 - ◇ Facilitate construction of various versions of an application
 - ◇ Defining and determining mechanisms for managing various versions of these work products
 - ◇ The functional baseline corresponds to the reviewed system requirements
 - ◇ Widely used baselines include functional, developmental, and product baselines
- ◇ In simple words, baseline means ready for release.

Baseline

- ◆ A baseline is a milestone in the development of software that marked the delivery of one or more software configuration items.

- ◆ IEEE define baseline as:

“A specification or product that has been formally reviewed and agreed upon, that thereafter serve as the basis for further development, and that can be changed only through formal change control procedure”

- ◆ Common Baselines :

- ◆ Software Engineering /Specification

- ◆ Requirement Analysis

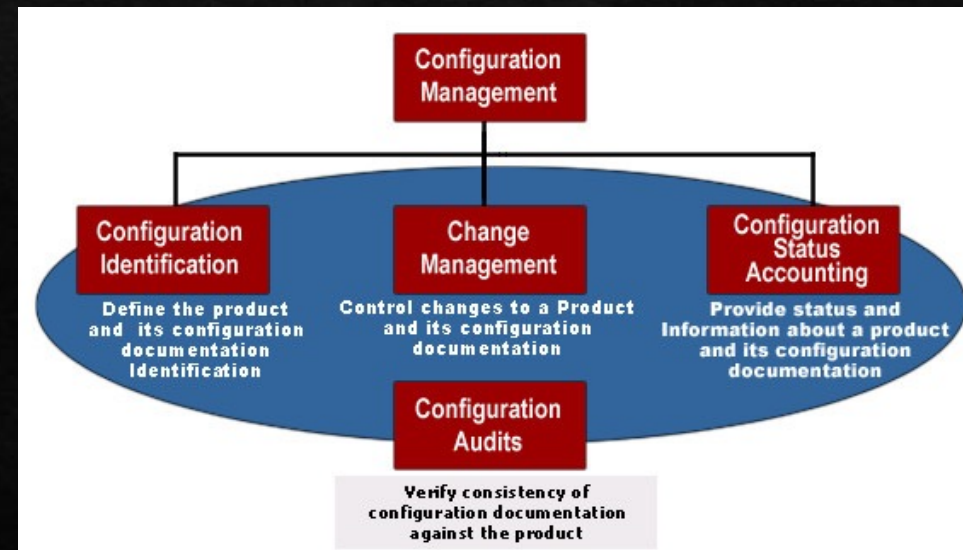
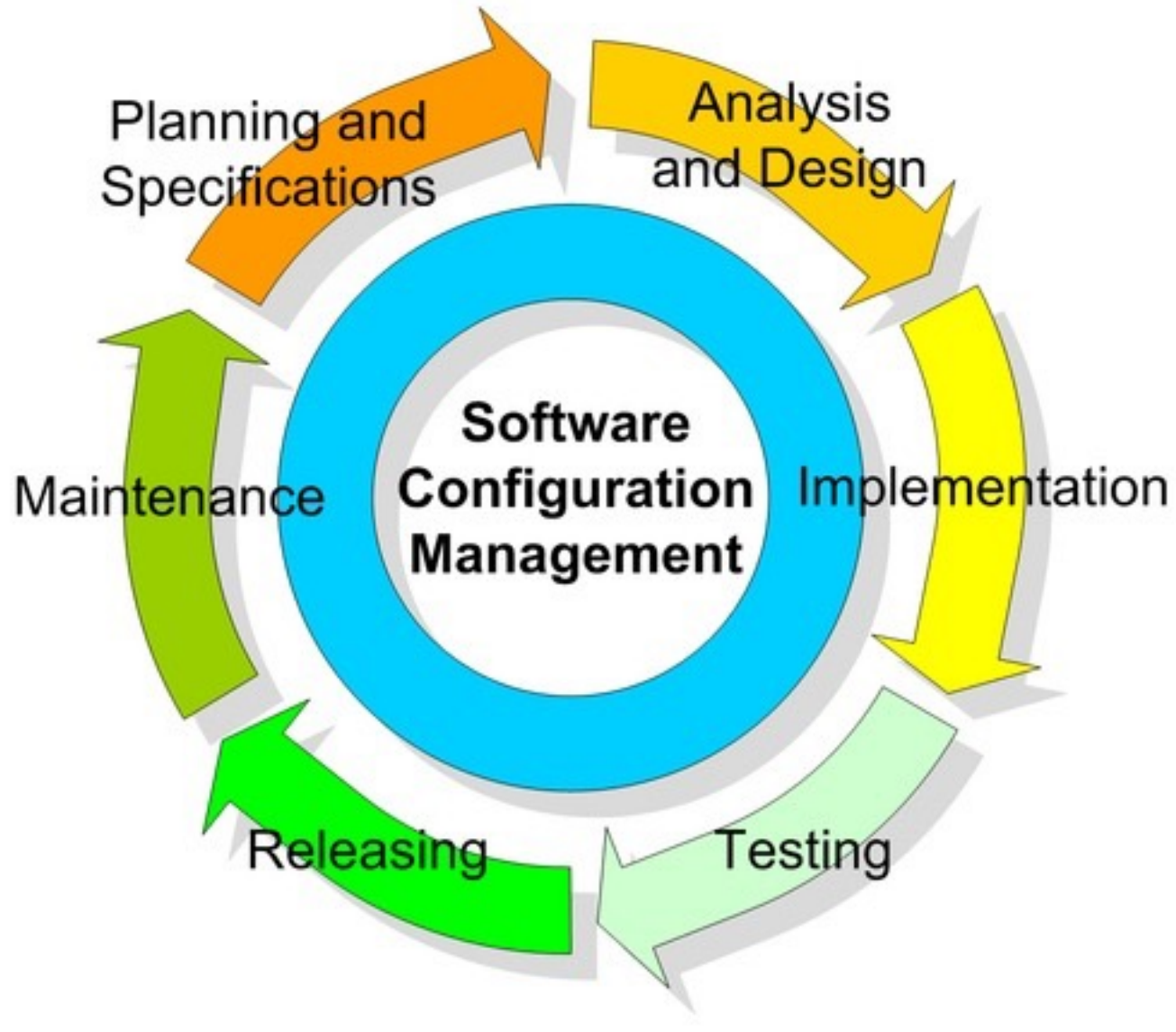
- ◆ Software design,

- ◆ Coding,

- ◆ Testing,

- ◆ Release

SCM Process



Participants of SCM Repository



1. Configuration Manager

- ◇ Configuration Manager is the head who is Responsible for identifying configuration items.
- ◇ CM ensures team follows the SCM process
- ◇ He/She needs to approve or reject change requests

2. Developer

- ◇ The developer needs to change the code as per standard development activities or change requests. He is responsible for maintaining configuration of code.
- ◇ The developer should check the changes and resolves conflicts

3. Auditor

- ◇ The auditor is responsible for SCM audits and reviews.
- ◇ Need to ensure the consistency and completeness of release.

4. Project Manager:

- ◇ Ensure that the product is developed within a certain time frame
- ◇ Monitors the progress of development and recognizes issues in the SCM process
- ◇ Generate reports about the status of the software system
- ◇ Make sure that processes and policies are followed for creating, changing, and testing

5. User

- ◇ The end user should understand the key SCM terms to ensure he has the latest version of the software

SCM stands for "Source Code Management" or "Software Configuration Management," and it refers to the management of changes to software code and related assets throughout the software development lifecycle.

SCM repositories are where the source code and related files are stored, versioned, and managed.

There are several types of SCM repositories, and two of the most common are:

1. Centralized Version Control System (CVCS):

2. Distributed Version Control System (DVCS):

There are several types of SCM repositories, and two of the most common are:

1. Centralized Version Control System (CVCS):

1. In a CVCS, a central server stores the entire history of the project, along with all the files and changes made by developers.
2. Developers work with a local working copy of the code. When they make changes, they commit those changes to the central repository.
3. Examples of CVCS include CVS (Concurrent Versions System) and Subversion (SVN).

2. Distributed Version Control System (DVCS):

1. In a DVCS, each developer has a complete copy of the repository, including the entire history.
2. Developers can commit changes locally, and these changes can later be pushed to other repositories or pulled from others.
3. This allows for more flexibility and collaboration among distributed teams.
4. Examples of DVCS include Git and Mercurial.

Git::

Git is currently one of the most widely used version control systems, and it is a distributed version control system. It is known for its speed, flexibility, and ability to handle both small and large projects effectively. Git repositories can be hosted on various platforms, such as GitHub, GitLab, Bitbucket, or even on a private server.

Here's a basic overview of how Git works:

- Developers **clone** a Git repository to create a local copy on their machine.
 - They **make changes** to the files in their **local copy**.
 - Changes are **committed** to their local repository, and a history of these changes is maintained.
 - Developers can **push** their changes to a central repository (hosted on a platform like GitHub) or pull changes made by others.
 - Git allows for branching and merging, enabling parallel development and collaboration.
- In summary, an SCM repository, often associated with Git, is a central place where the source code and its version history are stored. It facilitates collaboration, tracks changes, and provides a structured approach to managing software development projects.

Functions of an SCM Repository

- ◆ Data integrity → Validates entries, ensures consistency, cascades modifications
- ◆ Information sharing → Shares information among developers and tools, manages and controls multi-user access
- ◆ Tool integration → Establishes a data model that can be accessed by many software engineering tools, controls access to the data
- ◆ Data integration → Allows various SCM tasks to be performed on one or more CSCIs
- ◆ Methodology enforcement → Defines an entity-relationship model for the repository that implies a specific process model for software engineering
- ◆ Document standardization → Define objects in the repository to guarantee a standard approach for creation of software engineering documents

Change control

- ◇ Change control is a systematic approach that helps you manage incoming change requests throughout the life of your project.
- ◇ This approach focuses on thoroughly documenting and evaluating each change request to determine its feasibility and potential value. After this evaluation, the team can decide to approve, reject, or defer a project change until a later date.
- ◇ **Different Types of Change Requests**
 - ◇ Throughout the life of your project, there will be two kinds of changes that occur:
 - ◇ **Changes beyond your control:** weather events, team attrition, or employee sickness.
 - ◇ **Changes that are specifically requested:** Adding features to software applications, Updating network equipment, Installing system patches

Change control process

◆ Step 1: Document the Change Request

- ◆ When a change request occurs, your first step is to categorize and record it. This is required even if you ultimately decide not to pursue it.

◆ Step 2: Conduct a Formal Change Evaluation

- ◆ Your project team will meet and formally evaluate the change. Key questions to ask include:
- ◆ Is the change justified?
- ◆ What are the risks and benefits of making the change?
- ◆ What are the risks of not making the change?
- ◆ Do the risks outweigh the benefits?
- ◆ If you decide to accept a change, you'll assign the next steps to a dedicated development team.

◆ Step 3: Plan the Change

- ◆ The team that you task with developing the change will create a detailed plan outlining how it will be designed and implemented.
- ◆ At this juncture, there should be methods in place to define and verify success.

Change control process

◆ Step 4: Design Software Changes

- ◆ If the requested change affects your ERP software, the team will design the new program and test it for accuracy. If it passes successfully, they'll request approval and set a date for implementation.

◆ Step 5: Conduct an Internal Software Review

- ◆ If the software change is approved, the team will proceed to implement the change. Then, they'll perform demonstrations for stakeholders so they can review and accept the final change.

◆ Step 6: Conduct a Final Assessment

- ◆ If everything is implemented and reviewed successfully, the status of the change request will change from active to closed.